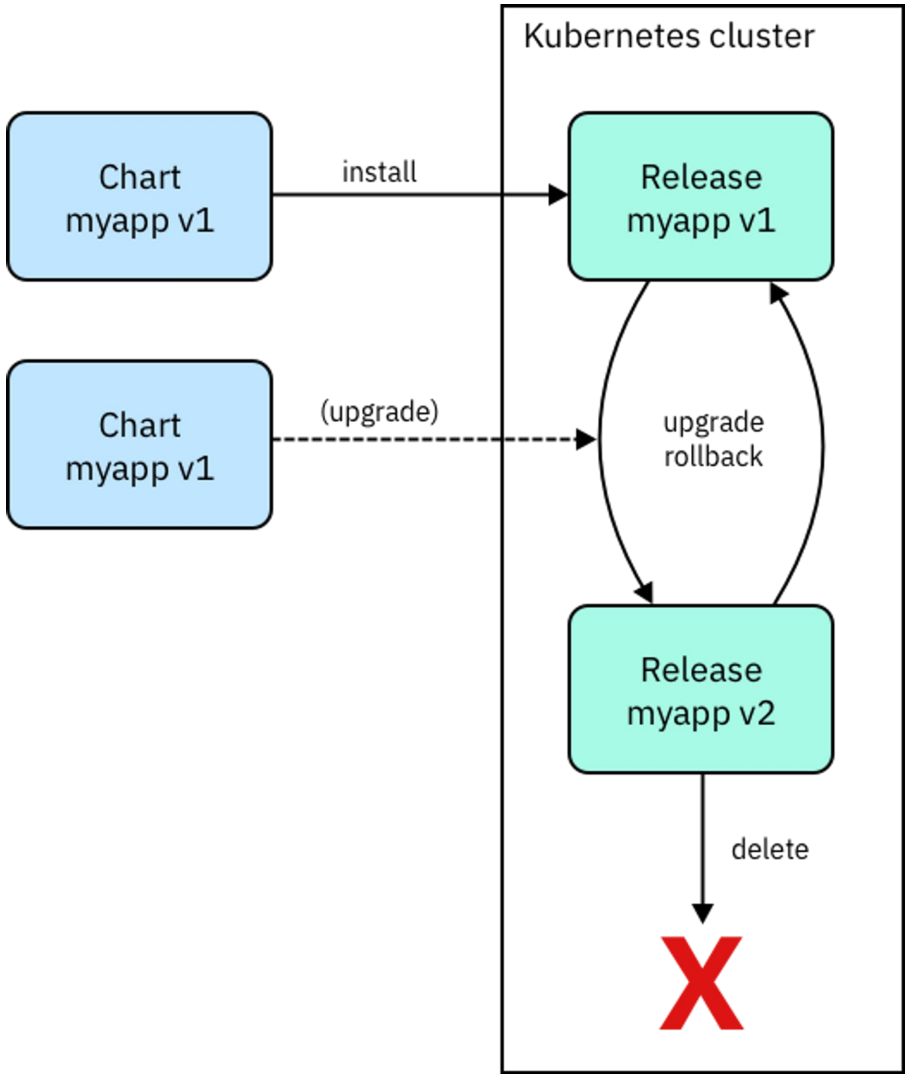
DevOps

Helm

What is Helm?

- Helm is a package manager for Kubernetes. Package managers automate the process of installing, configuring, upgrading, and removing computer programs. Examples include the Red Hat® Package Manager (RPM), Homebrew, and Windows® PackageManagement. It makes it easier to package and deploy software on a Kubernetes cluster using app definitions called charts.
- Helm Installation: https://helm.sh/docs/intro/install/



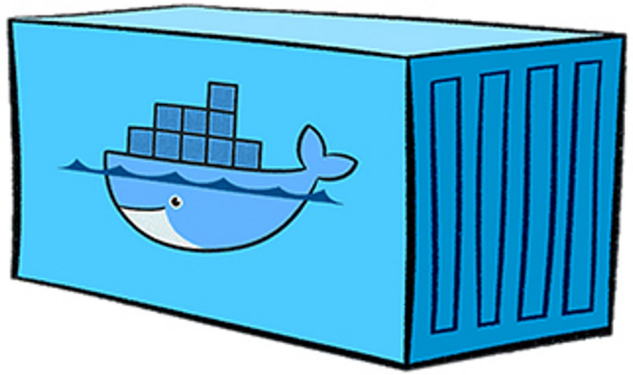
Helm Concept

- **Chart** is a package in Helm. It has a name, and contains a set of Kubernetes resource definitions and some templating logic that are used or can be shared and reused to deploy your application.

- **Repository** is an online collection of charts. It has a URL. You can search, download and install charts from a repository. The distributed community Helm chart repository is located at **Artifact Hub** and welcomes participation. But Helm also makes it possible to create and run your own chart repository.

Helm Concept

- **Release** is an instance or a deployment of a chart. When you perform a helm install command, you are creating a new release of that chart on your Kubernetes cluster.



Docker Container

Why is Helm?

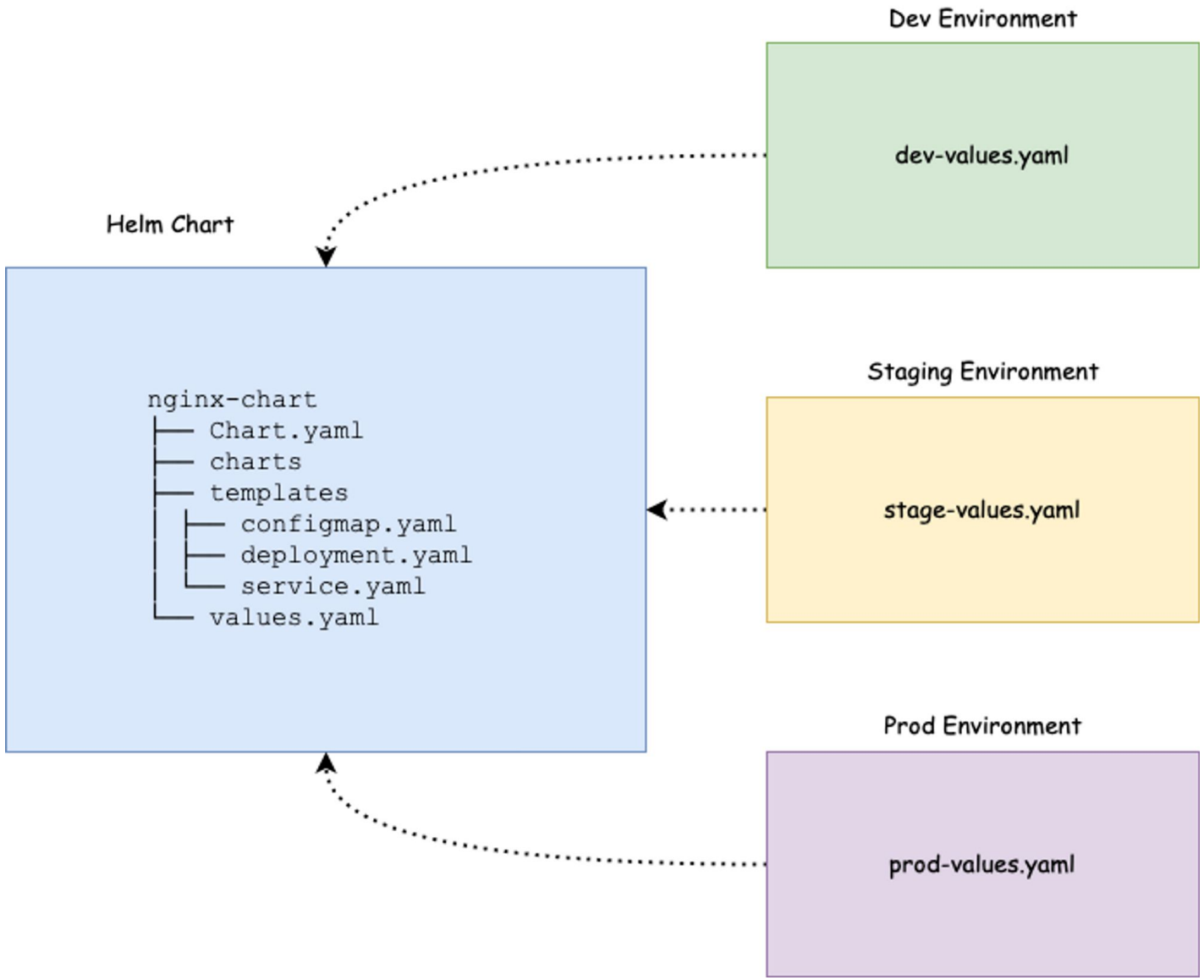
For the purpose of explanation, I am choosing a very basic example of a website frontend deployment using Nginx on Kubernetes

Let’s assume you have four different environments in your project. Dev, QA, Staging, and Prod. Each environment will have different parameters for Nginx deployment. For example,

- In Dev and QA you might need only one replica.
- In staging and production, you will have more replicas with pod autoscaling.
- The ingress routing rules will be different in each environment.
- The config and secrets will be different for each environment.

Because of the change in configs and deployment parameters for each environment, you need to maintain different Nginx deployment files for each environment. Or you will have a single deployment file and you will need to write custom shell or python scripts to replace values based on the environment. But it is not a scalable approach. Here is where the helm chart comes in to picture.

Why is Helm?

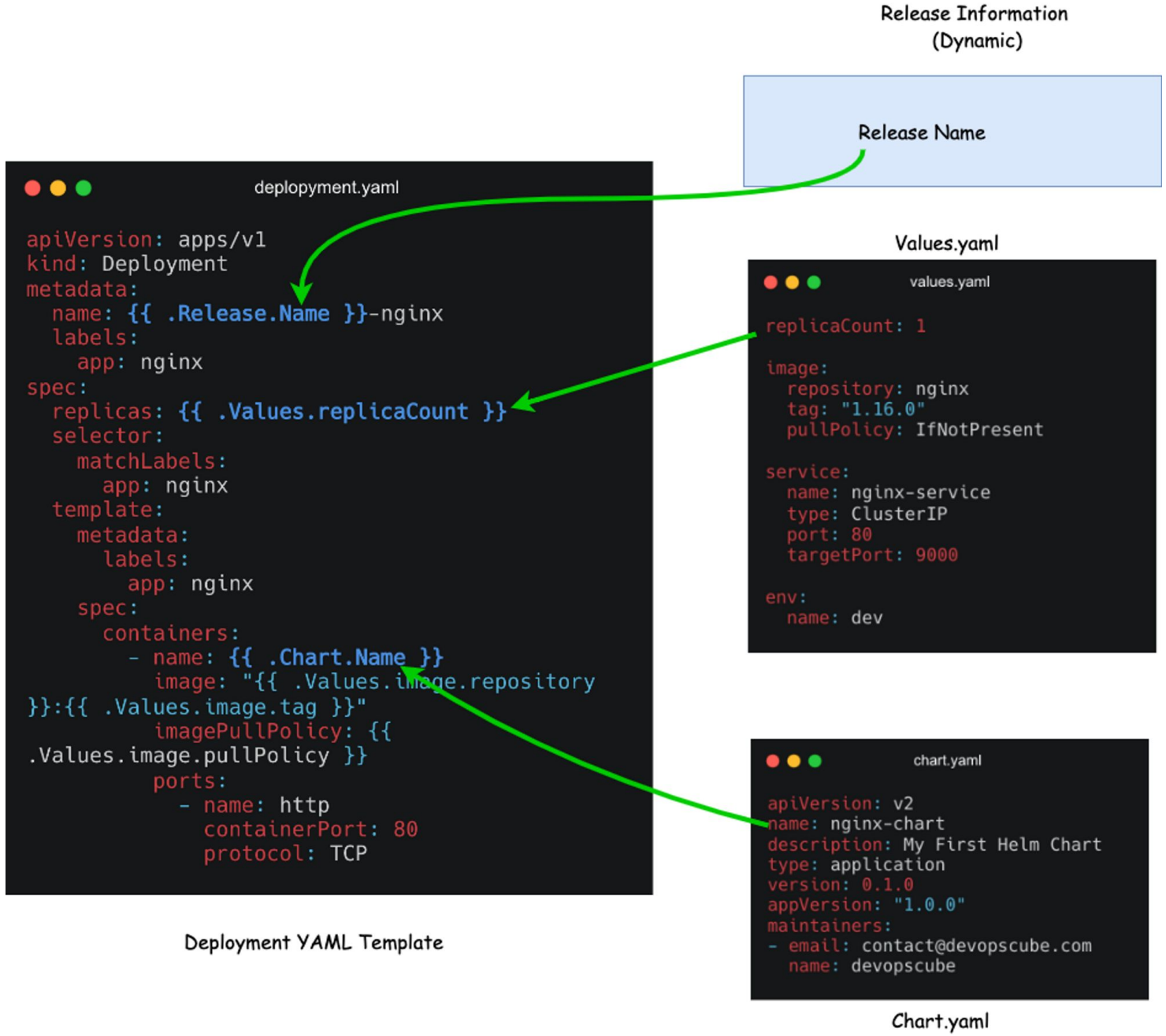

Dev Environment
dev-values.yaml
Helm Chart
nginx-chart
├── Chart.yaml
├── charts
├── templates
│ ├── configmap.yaml
│ ├── deployment.yaml
│ └── service.yaml
└── values.yaml
Staging Environment
stage-values.yaml
Prod Environment
prod-values.yaml

Why is Helm?

- `$ helm create nginx-chart`

```
nginx-chart/
|-- Chart.yaml
|-- charts
|-- templates
|   |-- NOTES.txt
|   |-- _helpers.tpl
|   |-- deployment.yaml
|   |-- configmap.yaml
|   |-- ingress.yaml
|   |-- service.yaml
|   `-- tests
|       `-- test-connection.yaml
`-- values.yaml
```

Why is Helm?


Release Information
(Dynamic)
Release Name
deplopyment.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
name: {{ .Release.Name }}-nginx
labels:
app: nginx
spec:
replicas: {{ .Values.replicaCount }}
selector:
matchLabels:
app: nginx
template:
metadata:
labels:
app: nginx
spec:
containers:
- name: {{ .Chart.Name }}
image: "{{ .Values.image.repository
}}:{{ .Values.image.tag }}"
imagePullPolicy: {{
.Values.image.pullPolicy }}
ports:
- name: http
containerPort: 80
protocol: TCP
Deployment YAML Template
Values.yaml
values.yaml
replicaCount: 1
image:
repository: nginx
tag: "1.16.0"
pullPolicy: IfNotPresent
service:
name: nginx-service
type: ClusterIP
port: 80
targetPort: 9000
env:
name: dev
chart.yaml
apiVersion: v2
name: nginx-chart
description: My First Helm Chart
type: application
version: 0.1.0
appVersion: "1.0.0"
maintainers:
- email: contact@devopscube.com
name: devopscube
Chart.yaml

Why is Helm?

- **.helmignore**: It is used to define all the files which we don't want to include in the helm chart. It works similarly to the .gitignore file.
- **Chart.yaml:** It contains information about the helm chart like version, name, description, etc.

```yaml
apiVersion: v2
name: nginx-chart
description: My First Helm Chart
type: application
version: 0.1.0
appVersion: "1.0.0"
maintainers:
- email: contact@devopscube.com
  name: devopscube
```

- **templates**: This directory contains all the Kubernetes manifest files that form an application. These manifest files can be templated to access values from values.yaml file. Helm creates some default templates for Kubernetes objects like deployment.yaml, service.yaml etc, which we can use directly, modify, or override with our files.

Why is Helm?

- **charts**: We can add another chart's structure inside this directory if our main charts have some dependency on others. By default this directory is empty.
- **templates/NOTES.txt**: This is a plaintext file that gets printed out after the chart is successfully deployed.

Why is Helm?

- **templates/_helpers.tpl**: a file that contains reusable template functions that can be used across multiple templates within the chart, and sub-template. These files are not rendered to Kubernetes object definitions but are available everywhere within other chart templates for use.
- **templates/tests/**: We can define tests in our charts to validate that your chart works as expected when it is installed.
- **values.yaml**: In this file, we define the values for the YAML templates. For example, image name, replica count, HPA values, etc. As we explained earlier only the values.yaml file changes in each environment. The **default values** for a deployment are stored in the **values.yaml** file in the chart. Also, you can override these values dynamically or at the time of installing the chart using **--values** or **--set** command.

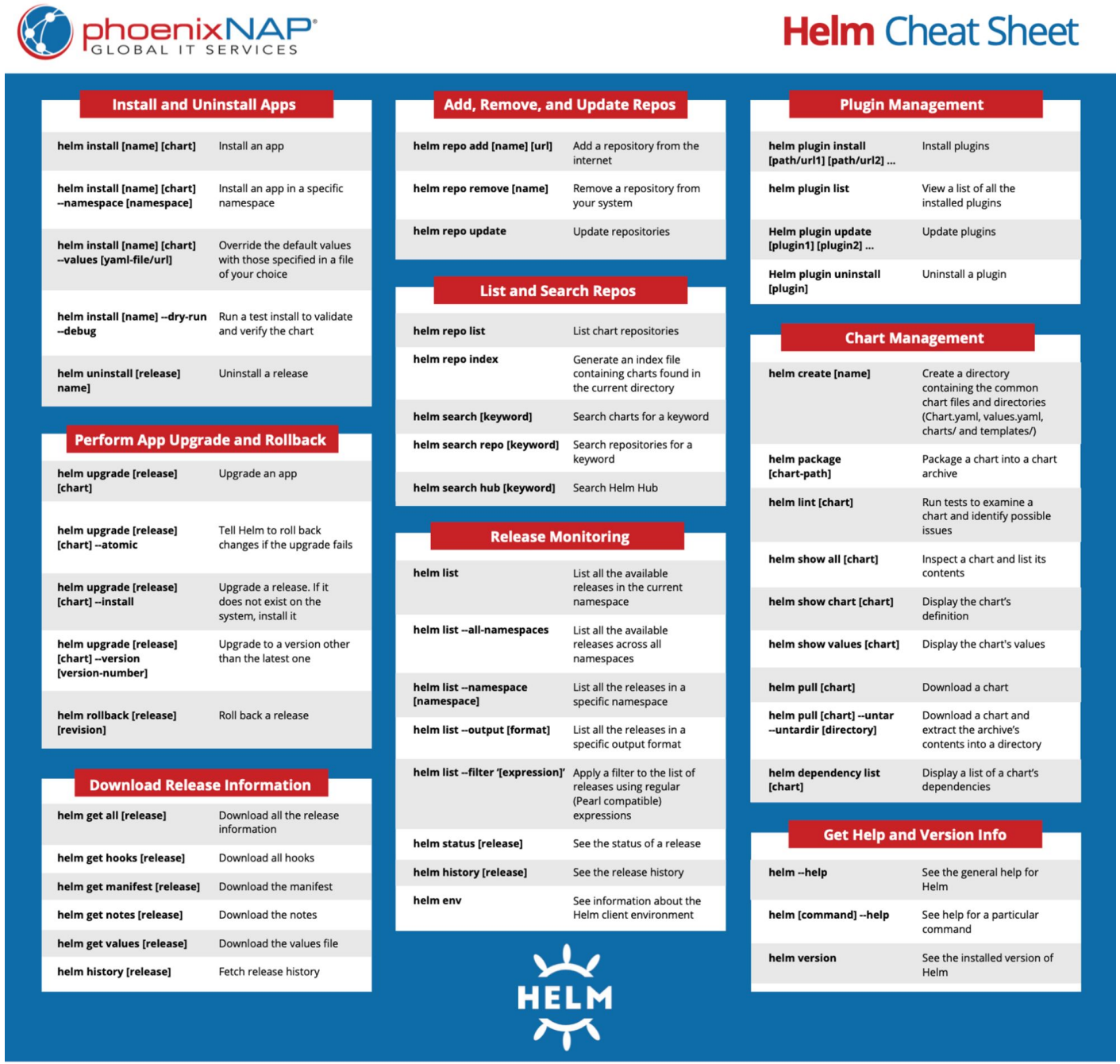
Helm Commands

```
$ helm version
$ helm create nginx-chart --namespace [namespace] # Generate chart template
$ helm lint nginx-chart # Check syntax
$ helm template nginx-chart # Preview chat template with values
$ helm show all nginx-chart # Show all chart info and values.yaml
$ helm show chart nginx-chart # Show chart info
$ helm install nginx-uat nginx-chart --values [yaml-file/url]
$ helm list -A # List all namespaces of releases
$ helm package nginx-chart # Package helm chart as tgz file
$ curl -u dara:123 https://nexus-ui.e-crops.co/repository/helm-repo/ --upload-file
nginx-chart-0.1.1.tgz # Upload to helm repository
$ helm repo add my-nginx https://nexus-ui.e-crops.co/repository/helm-repo/ --username dara
--password 123 # Download helm chart from repository
$ helm repo update # Update local repository
$ helm repo list # List all existing repository
$ helm install nginx-uat my-nginx/nginx-chart # Install helm chart from local repository
```

Helm Commands

```
$ tar xvf nginx-chart-0.1.0.tgz -C tmp # Extract file to tmp existing directory
$ tar xvf nginx-chart-0.1.0.tgz # Extract file here
$ helm install nginx-uat nginx-app-0.1.0.tgz # Install with .tgz file
$ helm uninstall nginx-uat # uninstall = delete
$ helm repo remove my-nginx # Remove repo from local repository
# The revision of a Helm release is increased whenever the release is upgraded or rolled back.
# Upgrade a release. If it does not exist on the system, install it
$ helm upgrade nginx-uat nginx-chart --install
$ helm upgrade nginx-uat nginx-chart --version 0.1.0 # Upgrade with specific chart version
$ helm rollback nginx-uat 2 # 2 is REVISION
$ helm get all nginx-chart # Show all info chart, deployment, service...
$ helm status nginx-uat # Check status of release
$ helm history nginx-uat # Show history of release
$ helm package postgres-chart --version 4.4.4 --app-version 1.2.3 # Package and Change Versions
$ helm create postgres-chart-new --starter=/home/leangsengk90/postgres-chart-5.5.5.tgz # or postgres-chart
$ helm fetch my-nginx/nginx-chart # Get helm chart file but not install
```

Helm Cheat Sheet



phoenixNAP GLOBAL IT SERVICES

Helm Cheat Sheet

Install and Uninstall Apps

helm install [name] [chart]	Install an app
helm install [name] [chart] --namespace [namespace]	Install an app in a specific namespace
helm install [name] [chart] --values [yaml-file/url]	Override the default values with those specified in a file of your choice
helm install [name] --dry-run --debug	Run a test install to validate and verify the chart
helm uninstall [release] name]	Uninstall a release

Perform App Upgrade and Rollback

helm upgrade [release] [chart]	Upgrade an app
helm upgrade [release] [chart] --atomic	Tell Helm to roll back changes if the upgrade fails
helm upgrade [release] [chart] --install	Upgrade a release. If it does not exist on the system, install it
helm upgrade [release] [chart] --version [version-number]	Upgrade to a version other than the latest one
helm rollback [release] [revision]	Roll back a release

Download Release Information

helm get all [release]	Download all the release information
helm get hooks [release]	Download all hooks
helm get manifest [release]	Download the manifest
helm get notes [release]	Download the notes
helm get values [release]	Download the values file
helm history [release]	Fetch release history

Add, Remove, and Update Repos

helm repo add [name] [url]	Add a repository from the internet
helm repo remove [name]	Remove a repository from your system
helm repo update	Update repositories

List and Search Repos

helm repo list	List chart repositories
helm repo index	Generate an index file containing charts found in the current directory
helm search [keyword]	Search charts for a keyword
helm search repo [keyword]	Search repositories for a keyword
helm search hub [keyword]	Search Helm Hub

Release Monitoring

helm list	List all the available releases in the current namespace
helm list --all-namespaces	List all the available releases across all namespaces
helm list --namespace [namespace]	List all the releases in a specific namespace
helm list --output [format]	List all the releases in a specific output format
helm list --filter '[expression]'	Apply a filter to the list of releases using regular (Pearl compatible) expressions
helm status [release]	See the status of a release
helm history [release]	See the release history
helm env	See information about the Helm client environment

HELM

Plugin Management

helm plugin install [path/url1] [path/url2] ...	Install plugins
helm plugin list	View a list of all the installed plugins
Helm plugin update [plugin1] [plugin2] ...	Update plugins
Helm plugin uninstall [plugin]	Uninstall a plugin

Chart Management

helm create [name]	Create a directory containing the common chart files and directories (Chart.yaml, values.yaml, charts/ and templates/)
helm package [chart-path]	Package a chart into a chart archive
helm lint [chart]	Run tests to examine a chart and identify possible issues
helm show all [chart]	Inspect a chart and list its contents
helm show chart [chart]	Display the chart's definition
helm show values [chart]	Display the chart's values
helm pull [chart]	Download a chart
helm pull [chart] --untar --untardir [directory]	Download a chart and extract the archive's contents into a directory
helm dependency list [chart]	Display a list of a chart's dependencies

Get Help and Version Info

helm --help	See the general help for Helm
helm [command] --help	See help for a particular command
helm version	See the installed version of Helm

Practice 01

```
nginx-chart/
├── Chart.yaml
├── charts
├── templates
│   ├── deployment.yml
│   └── service.yml
├── values-prod.yaml
└── values.yaml
```

Practice

```
# templates/deployment.yml
{{- if gt (int $.Values.replicas) 3 }}
apiVersion: apps/v1
kind: Deployment
metadata:
 name: {{ .Values.appName.app }}-app
spec:
 selector:
   matchLabels:
     {{- toYaml .Values.appName | nindent 6 }}
 replicas: {{ .Values.replicas }}
 template:
   metadata:
     labels:
       {{- toYaml .Values.appName | nindent 8 }}
   spec:
     containers:
     {{- with .Values.container }}
     - name: {{ .name }}
       image: {{ .image }}
       ports:
       - containerPort: {{ .port }}
     {{- end }}
{{- else if le (int $.Values.replicas) 3 }}
 {{- fail "Replicas <= 3" }}
{{- end }}
```

```
# templates/service.yml
{{- if .Values.appName.app }}
apiVersion: v1
kind: Service
metadata:
 name: {{ .Values.appName.app }}-svc
spec:
 selector:
   {{- toYaml .Values.appName | nindent 4 }}
 ports:
   - protocol: TCP
     port: {{ .Values.container.port }}
     targetPort: {{ .Values.container.port }}
{{- else }}
 {{- fail "Something went wrong" }}
{{- end }}
```

Practice

```yaml
# values.yaml
appName:
 app: nginx
replicas: 5
container:
 name: nginx-cont
 image: nginx:1.22.1
 port: 80
```

```yaml
# values-prod.yml
appName:
 app: nginx-prod
replicas: 4
container:
 name: nginx-prod-cont
 image: nginx:latest
 port: 80
```

```
$ helm install nginx-uat nginx-chart
$ helm install nginx-prod nginx-chart --values nginx-chart/values-prod.yaml
```

Practice 02

```
nginx-chart
├── charts
├── Chart.yaml
├── templates
│   ├── deployment
│   │   └── nginx-deploy.yml
│   ├── NOTES.txt
│   ├── service
│   │   └── nginx-service.yml
│   ├── tests
│   │   └── test.yml
│   └── tpl
│       ├── deployment.tpl
│       └── _helpers.tpl
└── values.yaml
```

Practice: tests

```yaml
# templates/tests/test.yml
apiVersion: v1
kind: Pod
metadata:
 name: pod-test
 annotations:
   "helm.sh/hook": test
spec:
 containers:
   - name: busybox-cont
     image: busybox
     command: ['telnet']
     args: ['nginx-svc', '80']
 restartPolicy: Never
```

- Add these lines to Chart.yml

```yaml
test:
    enabled: true
    template: tests/test.yml
```

```
$ helm test nginx-uat
```

Practice: tests

```
leangsengk90@node1:~/my-helm/nginx-chart$ helm test nginx-uat --logs
NAME: nginx-uat
LAST DEPLOYED: Sat Jun 17 05:55:48 2023
NAMESPACE: default
STATUS: deployed
REVISION: 1
TEST SUITE:     pod-test
Last Started:   Sat Jun 17 05:58:05 2023
Last Completed: Sat Jun 17 05:58:13 2023
Phase:          Failed
NOTES:
===============
***  USAGE  ***
===============

#Get all Services
$ kubectl get all

#Get Pod detail
$ kubectl describe pod pod-name -n namespace

POD LOGS: pod-test
telnet: can't connect to remote host (10.233.11.55): Connection refused

Error: 1 error occurred:
        * pod pod-test failed
```

Failure

```
POD LOGS: pod-test
Connected to nginx-svc
```

Success

Practice: _helpers.tpl

```
# templates/tpl/deployment.tpl
{{- define "deployment.tpl" }}
apiVersion: apps/v1
kind: Deployment
metadata:
 name: {{ .Values.nginxApp.name }}-deploy
spec:
 replicas: {{ .Values.nginxApp.replicas }}
 selector:
   matchLabels:
     app: {{ .Values.nginxApp.name }}-app
 template:
   metadata:
     labels:
       app: {{ .Values.nginxApp.name }}-app
   spec:
     containers:
       - name: {{ .Values.nginxApp.name }}-cont
         image: {{ .Values.nginxApp.image }}:{{ .Values.nginxApp.tag }}
         ports:
           - containerPort: {{ .Values.nginxApp.containerPort }}
{{- end }}
```

Practice: _helpers.tpl

```
# templates/tpl/_helpers.tpl
{{- define "selector" }}
 selector:
   app: {{ .Values.nginxApp.name }}-app
{{- end }}
{{- define "ports" -}}
ports:
 - port: {{ .Values.nginxApp.port }}
   targetPort: {{ .Values.nginxApp.targetPort }}
{{- end -}}
```

Practice: _helpers.tpl

```
# templates/values.yml
nginxApp:
  name: "nginx"
  image: "nginx"
  tag: "1.22.1"
  replicas: 1
  port: 80
  targetPort: 80
  containerPort: 80
```

```
# templates/deployment/nginx-deployment.yml
{{- template "deployment.tpl" . }}
```

```
# templates/service/nginx-service.yml
apiVersion: v1
kind: Service
metadata:
 name: {{ .Values.nginxApp.name }}-svc
spec:
 type: ClusterIP
{{- template "selector" . -}}
{{ include "ports" . | nindent 2 }}
```

Practice: Loop

```
# templates/postgres-deployment.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
 name: {{ .Values.postgres.name }}
spec:
 replicas: 1
 selector:
   matchLabels:
     app: {{ .Values.postgres.name }}
 template:
   metadata:
     labels:
       app: {{ .Values.postgres.name }}
   spec:
     containers:
       - name: {{ .Values.postgres.name }}
         image: {{ .Values.postgres.image }}
         env:
           {{- range .Values.postgres.env }}
           - name: {{ .name }}
             value: "{{ .value }}"
           {{- end }}
         ports:
           - containerPort: {{ .Values.postgres.port }}
```

Practice: Loop

```yaml
# values.yaml
postgres:
 name: postgres
 image: postgres
 port: 5432
 env:
   - name: POSTGRES_PASSWORD
     value: "123"
   - name: POSTGRES_DB
     value: "test"
   - name: POSTGRES_USER
     value: "dara"
```

Thank you

Begin with the theory, Start with the practical